Limits to Single Slot Finality

Ethereum's timing constraints, from EIP-5133 to fast finality

Abhimanyu Nag

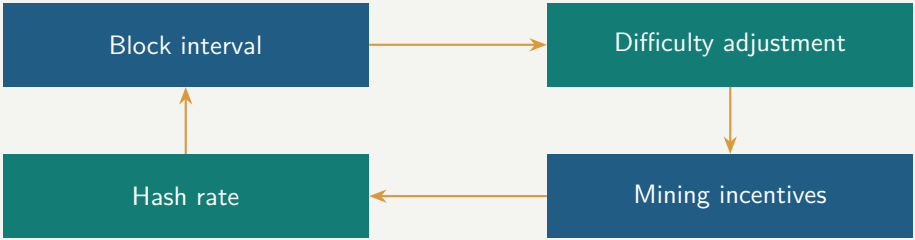
University of Alberta

EIP-5133: keeping Ethereum usable before the Merge

- 2022
- Delay the difficulty bomb by 700,000 blocks
- Bomb offset: 10.7M to 11.4M blocks
- Protect block production while proof of stake approaches

Source: Official EIPs, status checked 16 Oct 2026

The difficulty bomb was a timing feedback problem



The lesson: protocol time depends on assumptions about the system

Source: Primary source, checked 16 Oct 2026

EIP-5133 in Geth

```
calcDifficultyEip5133 = makeDifficultyCalculator(big.NewInt(11_400_000))

// CalcDifficulty: select rules for the next block
case config.IsGrayGlacier(next):
    return calcDifficultyEip5133(time, parent)
```

- The fork selects a new bomb offset, while keeping the difficulty calculator
- The calculator accounts for the parent being one block earlier

Source: Geth v1.10.20

Block production and finality use different clocks

Clock	What it measures	What it does not establish
Inclusion	A transaction enters a block	That the block will survive
Confirmation	A protocol-specific confidence claim	Always accountable finality
Finality	A committed ledger prefix	Unconditional safety under any fault

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Gasper separates the available head from the finalized prefix

- LMD-GHOST chooses the head
- Casper FFG finalizes checkpoints



32 slots per epoch in the baseline design
64–95 slots is roughly 13–19 minutes at 12 s/slot

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

The finality target must include a fault model

Safety

Honest nodes never finalize conflicting histories under the fault bound

Liveness

Finality resumes with sufficient honest participation after the network stabilizes

Dynamic availability

The available chain can progress as participation changes under its own assumptions

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

An attestation carries data and participant metadata

```
# Electra container, field types abbreviated
class Attestation(Container):
    aggregation_bits: Bitlist[...]
    data: AttestationData
    signature: BLSSignature
    committee_bits: Bitvector[...]
```

- Signed data carries the head root and source/target checkpoints
- Participant bitfields identify whose effective balances count

Source: consensus-specs v1.6.0

Fork choice validates a vote before updating weight

```
# on_attestation: condensed specification flow
validate_on_attestation(store, attestation, is_from_block)
store_target_checkpoint_state(store, attestation.data.target)
state = store.checkpoint_states[attestation.data.target]
indexed = get_indexed_attestation(state, attestation)
assert is_valid_indexed_attestation(state, indexed)
update_latest_messages(store, indexed.attesting_indices,
                       attestation)
```

- Unknown blocks and vote timing need explicit handling
- get_head follows viable branches weighted by latest messages

Source: consensus-specs v1.6.0

Finality processing is epoch-based in this baseline

```
# weigh_justification_and_finalization: selected branches
if current_epoch_target_balance * 3 >= total_active_balance * 2:
    state.current_justified_checkpoint = Checkpoint(
        epoch=current_epoch, root=get_block_root(state, current_epoch))
    state.justification_bits[0] = 0b1

if all(bits[0:2]) and old_current_justified_checkpoint.epoch + 1 == current_epoch:
    state.finalized_checkpoint = old_current_justified_checkpoint
```

- The threshold uses effective balance, not a count of signatures
- Finalization also depends on checkpoint history and justification bits

Source: consensus-specs v1.6.0

Quorum intersection creates accountable safety

$$w(Q_1) \geq \tfrac{2}{3}, \quad w(Q_2) \geq \tfrac{2}{3} \quad \Longrightarrow \quad w(Q_1 \cap Q_2) \geq \tfrac{1}{3}$$

- Conflicting finalized histories require slashable voting under the protocol's rules
- Weight refers to stake. Voting rules establish slashability.

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Availability and finality have different partition behavior

- An available chain may keep producing blocks
- The finalized prefix may stop advancing
- After recovery, the protocol must reconnect these ledgers safely

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Goldfish and RLMD-GHOST manage the age of votes

Goldfish

Buffer messages and expire votes to align honest views

RLMD-GHOST

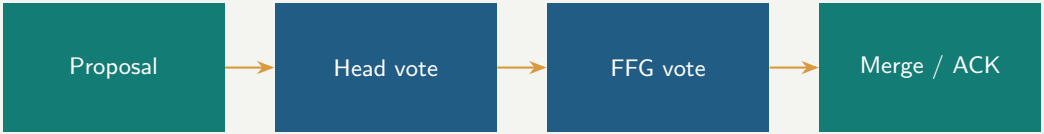
Retain votes over a tunable window η

Design choice

Longer memory changes asynchrony resilience and participation assumptions

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

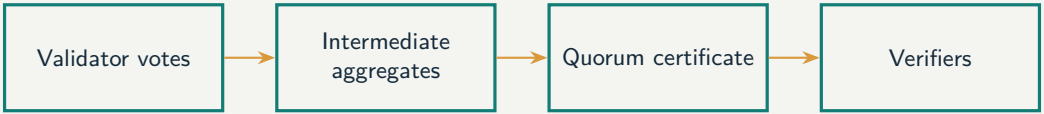
Single-slot finality puts dependent votes inside one slot



Same-slot finality requires the dependent evidence to arrive before the slot ends

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Aggregation has an upstream path and a downstream path



Collection and construction precede delivery and verification.

A compact certificate can coexist with expensive certificate production

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Lighthouse stages verification and avoids duplicate work

```
// Conceptual flow using the source's actual function names
IndexedAggregatedAttestation::verify(...)
verify_signed_aggregate_signatures(...)
VerifiedAggregatedAttestation::verify_late_checks(...)

// Inside verify_signed_aggregate_signatures:
Ok(verify_signature_sets(signature_sets.iter()))
```

- The batch covers selection proof, outer signature and aggregate signature
- Late checks record observations and reject already-known subsets

Source: Lighthouse stable snapshot, 16 Oct 2026

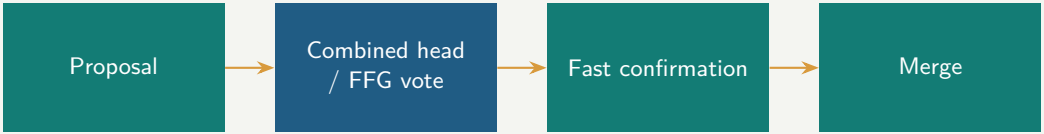
The slot budget follows the critical path

$$T_{\text{slot}} \geq T_{\text{proposal}} + rT_{\text{vote}} + T_{\text{other}}$$

- Tvote includes collection, aggregation and dissemination
- r counts dependent voting rounds
- Overlapping work must be counted only once

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

3SF reduces the number of voting rounds per slot



One critical voting round per slot. Three-slot finality requires synchrony and sufficient honest participation.

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Shorter slots and earlier finality are different objectives

Model	SSF slot	3SF slot
Unit-delay schematic	4Δ	4Δ
Aggregation-aware illustration	6Δ	5Δ
Honest proposal finalization (aggregation-aware)	5Δ	11Δ

Fewer aggregation rounds can shorten slots while increasing finality latency

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Validator keys and independent operators are different counts

$$k(B_{\max}) = \sum_i \left\lceil \frac{s_i}{B_{\max}} \right\rceil \geq \frac{S}{B_{\max}}$$

- Raising the cap can reduce required keys at fixed operator stakes
- Consolidation leaves stake concentration unchanged in this model

s_i: stake of operator i. S: total stake. k: minimum required keys.

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

The paper's bandwidth bound is conditional

$$qkb \leq W\tau$$

- q: required fraction of keys in the model
- b: per-key information cost at the measured receiver
- W: effective receive rate, τ: available time
- If q $\cdot$ k $\cdot$ b $>$ W $\cdot$ τ, the required receipt cannot fit within τ under this model

The bound applies where the additive per-key receipt-cost assumption holds

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

A numerical example shows sensitivity, not a latency threshold

- Illustration: $k = 1{,}000{,}000$ and $q = 2/3$
- 96-byte contribution at one receiver: 64 MB per round
- At 100 Mbit/s effective ingress: at least 5.12 s
- Signer bitmap alone: 125 kB for one million keys

Topology and certificate design determine where the work accumulates

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Global gossip needs a tail-latency budget

Propagation

Peer hops, geographic delay and payload serialization

Local work

Queueing, signature checks and aggregate construction

Adversarial stress

Withheld votes, equivocation and overloaded aggregators

A safe deadline needs measured quorum-arrival tails and a failure policy

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

EIPs that reduce the aggregation workload

EIP	Mechanism	Relation to fast finality
7251 · Final	Cap rises to 2,048 ETH Minimum activation stays 32 ETH	Enables fewer keys through consolidation
7549 · Final	Committee index leaves signed data	Allows aggregation of otherwise equal votes

Source: Official EIPs, status checked 16 Oct 2026

Recent timing and validation proposals

Proposal	Verified status	What it changes
7782	Draft	Six-second slots and tighter attestation timing
7732	Last Call	Separates consensus and execution validation
7942	Draft	Attestation-based stability and reorg resilience

Source: Official EIPs, status checked 16 Oct 2026

Implementing SSF changes more than slot duration

Component	Required design work
State and messages	Slot-level checkpoints, vote roles and quorum evidence
Networking	Collection deadlines, buffering, late-vote and retry policy
Safety	Slashing conditions and fork-choice/finality compatibility
Validation	State-transition vectors, adversarial traces and client interop

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

The research frontier extends beyond EIP numbers

Lean consensus

Executable specifications and multi-client experiments

New cryptography

Hash-based signatures and recursive aggregation research

Decoupled finality

Fast available blocks with a separate finality cadence

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

The open problem is an end-to-end feasible protocol

Proof obligation

Safety, liveness and participation assumptions for the full design

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Engineering obligation

Quorum arrival within the deadline on realistic nodes

Economic obligation

Participation remains viable across operators and geographies

Limits to Single Slot Finality

- Network and participation assumptions constrain the guarantees
- Aggregation design determines which workload reaches the critical path
- 3SF trades finality depth for a shorter slot budget
- Recent EIPs improve components of that budget

Source: Nag and Saraswat, ICBC 2026 (supplied paper)

Discussion

- What finality guarantee should Ethereum optimize for?
- Which bottleneck dominates on a realistic global deployment?
- How much work can move off the block-production path?

Source: Nag and Saraswat, ICBC 2026 (supplied paper)